

What is RAG?

RAG (Retrieval-Augmented Generation) enhances LLMs by retrieving relevant external knowledge before generating.

Key Benefits:

- Reduces hallucinations with grounded responses
- Enables real-time, up-to-date information access
- Domain-specific knowledge without fine-tuning
- Transparent & auditable (source attribution)

RAG vs Fine-tuning

Aspect	RAG	Fine-tuning
Data freshness	Real-time	Static
Cost	Lower	Higher
Latency	Higher	Lower
Transparency	High	Low
Updates	Instant	Retrain

Core RAG Components

- Document Loader: Ingest PDFs, web, DBs
- Text Splitter: Chunk documents
- Embedding Model: Text → vectors
- Vector Store: Index embeddings
- Retriever: Find relevant chunks
- Reranker: Refine results
- LLM: Generate response

RAG Pipeline Architecture

INDEXING PHASE (Offline)



QUERY PHASE (Online)



GENERATION PHASE



RAG Paradigms

Naïve RAG	Basic retrieve → generate
Advanced RAG	Pre/post retrieval optimization
Modular RAG	Flexible components
Agentic RAG	LLM decides strategy
Graph RAG	Knowledge graph integration
Self-RAG	Self-reflection & critique

Core Formulas

$$\text{sim}(q, d) = \cos(E(q), E(d))$$

Cosine Similarity for Vector Search

$$\text{score} = a * \text{dense} + (1 - a) * \text{sparse}$$

Hybrid Search Fusion ($a=0.5-0.7$)

Chunking Strategies

Method	Use Case	Notes
Fixed Size	General purpose	Simple, may cut cont
Recursive	Structured docs	Hierarchical splitti
Semantic	Topic-based	Meaning-aware, slowe
Sentence	Q&A systems	Natural boundaries
Document	Long-form	Headers, sections
Agentic	Complex docs	LLM-driven, costly

Chunk Size Guidelines

- Small (128-256): Precise retrieval, Q&A
- Medium (512-1024): Balanced, general use
- Large (1024-2048): Context-heavy tasks
- Overlap: 10-20% of chunk size
- Start: 512 tokens, 50 overlap

LangChain Chunking

```
from langchain.text_splitter import \
    RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512,
    chunk_overlap=50,
    separators=['\n\n', '\n', '. ']
)
chunks = splitter.split_documents(docs)
```

Advanced Chunking

- **Parent-Child:** Small linked to large parent
- **Late Chunking:** Embed full, chunk after
- **Contextual:** Add summary to each chunk
- **Proposition:** Extract atomic facts

Embedding Models

Model	Dims	Ctx	Score
text-embedding-3-lar	3072	8K	64.6
text-embedding-3-sma	1536	8K	62.3
voyage-3	1024	32K	67.8
cohere-embed-v3	1024	512	64.5
jina-embeddings-v3	1024	8K	65.4
bge-large-en-v1.5	1024	512	64.2
e5-mistral-7b	4096	32K	66.6
mxbai-embed-large	1024	512	64.7

Arabic Embedding Models

- **CAMeL-BERT:** MSA + Dialectal, 768d
- **AraBERT v2:** MSA focused, 768d
- **Jais-13b:** Arabic-English bilingual
- **multilingual-e5:** 100+ languages, 1024d
- **acge_text_embedding:** Arabic optimized, 1792d

Embedding Best Practices

- Same model for queries & documents
- Instruction-tuned for better queries
- Normalize vectors for cosine sim
- Batch embed (batch_size=32)
- Cache to reduce API costs
- Test domain-specific vs general
- For Arabic: use multilingual models

OpenAI Embeddings

```
from openai import OpenAI
client = OpenAI()

def embed(text):
    resp = client.embeddings.create(
        input=text,
        model='text-embedding-3-small'
    )
    return resp.data[0].embedding
```

Vector Databases

Database	Type	Best For
Pinecone	Managed	Production scale
Weaviate	Open/Cloud	Hybrid search
Qdrant	Open-source	High performance
Milvus	Open-source	Enterprise
Chroma	Open-source	Prototyping
pgvector	Extension	PostgreSQL
Redis	In-memory	Low latency
LanceDB	Embedded	Local/edge

Vector Index Types

- **Flat:** Exact search $O(n)$, small data
- **IVF:** Inverted file, partitioned
- **HNSW:** Graph-based, fast & accurate
- **PQ:** Product quantization, compressed
- **DiskANN:** SSD-optimized, billion scale

Distance Metrics

- Cosine: Direction-based (default for text)
- Dot Product: Magnitude matters
- Euclidean (L2): Absolute distance
- Use cosine for normalized embeddings

LangChain + Pinecone

```
from langchain_pinecone import PineconeVectorStore
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()
vectorstore = PineconeVectorStore(
    index_name='my-index',
    embedding=embeddings
)
retriever = vectorstore.as_retriever(k=5)
```

Retrieval Strategies

- Dense Retrieval**
Semantic similarity via embeddings
+ Understands meaning - Misses exact matches
Best for: semantic search, Q&A
- Sparse Retrieval (BM25)**
Keyword/term frequency matching
+ Fast, interpretable - No semantics
Best for: exact keyword search
- Hybrid Search**
Combines dense + sparse scores
+ Best of both Uses RRF or weighted
Best for: production systems

Reranking Models

- **Cohere Rerank:** Best accuracy, API
- **bge-reranker-v2-m3:** Open, multilingual
- **cross-encoder:** Classic, well-tested
- **RankGPT:** LLM-based reranking

Advanced Retrieval Techniques

- **MMR:** Maximal Marginal Relevance - diversity
- **HyDE:** Hypothetical doc embedding
- **Multi-Query:** Generate query variations
- **Step-back:** Abstract query for reasoning
- **Self-Query:** LLM extracts metadata filters
- **Parent Doc:** Retrieve parent of match

Advanced RAG Architectures

- RAPTOR**
Recursive tree summarization
Multi-level retrieval, good for long docs
- GraphRAG**
Knowledge graph + embeddings
Entity relations, complex queries
- Self-RAG**
LLM self-reflection
Critic tokens, reduces hallucination
- CRAG**
Corrective RAG
Confidence eval, web fallback
- Fusion RAG**
Multi-source merging
Combines multiple retrievers

Query Transformation

- Rewriting: Clarify ambiguous queries
- Expansion: Add synonyms/related terms
- Decomposition: Break into sub-queries
- HyDE: Generate hypothetical answer first
- Step-back: Ask abstract question

Evaluation Metrics (RAGAS)

- Faithfulness** 0.85+
Grounded in context?
- Answer Relevancy** 0.80+
Addresses question?
- Context Precision** 0.75+
Chunks relevant?
- Context Recall** 0.80+
All info retrieved?
- Correctness** 0.85+
Factually accurate?

Additional Metrics

- **MRR:** Mean Reciprocal Rank
- **NDCG:** Normalized DCG
- **Hit Rate:** % relevant in top-k
- **Latency:** P50, P95, P99 times

Production Checklist

- ✓ Implement hybrid search
- ✓ Add reranking step
- ✓ Cache embeddings & queries
- ✓ Use metadata filtering
- ✓ Monitor with RAGAS metrics
- ✓ Implement fallbacks
- ✗ One-size-fits-all chunks
- ✗ Skip evaluation
- ✗ Ignore latency